We assume the input for the algorithm is a text, stored in a string (of course, the actual text might be stored in a file or stream, but we can always have a way to convert it to a string[17]), and the output is a map from characters to binary sequences.

The first sub-task that we need to perform is to transform the text: we want to compute some statistics on it, to understand what are the most used and least used characters in it. To that end, we compute the frequency of characters in the text[18].

The details of the `ComputeFrequencies` method at line #1 are both out of scope and (at least in its basic version) simple enough, and there is no need to delve into that helper method here.

Once we have computed the frequency map, we create a new priority queue and then at lines #3 and #4 we iterate over the frequency map, creating a new `TreeNode` for each character and then adding it to the priority queue. Obviously, considering the subject of this chapter, for the queue we use a heap, and in particular a min-heap, where the element at the top is the one with the smallest value for the priority field. And in this case the priority field is (not surprisingly) the frequency field of the `TreeNode`.

Listing 2.14 The Huffman Coding Algorithm

```
function huffman(text)
  charFrequenciesMap ←
ComputeFrequencies(text)                          #1
```